

Product name

MAISON Fashion & Apparel E-commerce

Domain category

Clothing Shop

Team name

Alphamonno1

Members

Peerawit Yawirach	682115032
-------------------	-----------

TingFeng Wang	682115509
---------------	-----------

Netipong Saiso	682115023
----------------	-----------

Deployment URL: <https://clothprojectrender.onrender.com> (/admin is the administrator system)

GitHub Repository URL:

<https://github.com/camt-pathathai/953261-953202-term-project-alphamonno1.git>

Course: 953202 Introduction to Software Engineering

953261 Interactive Web Development

Academic Year: 2/2025

Table of Contents

Update this section if you are generating a PDF.

Table of Contents	2
1. Team Information	3
2. Product Vision	4
3. Scope and MVP Requirements	4
3.1 Customer Features (MVP)	4
3.2 Admin Features (MVP)	5
3.3 Optional Feature (Choose exactly 1)	5
3.4 Out of Scope (Not required)	5
4. Use Cases	5
Use Case: Browse products	6
Use Case: Checkout & create order	6
Use Case: Admin manages products	7
5. System Architecture (MVC)	7
5.1 Technology Stack (fixed)	7
5.2 Tools	8
6. Software Process Model	8
7. Project Timeline (3 Weeks)	8
8. Git Collaboration Rules (No PR)	8
9. UI Design (Lightweight)	9
9.1 Low-Fidelity Wireframes	11
9.2 Logo and Color theme	11
9.3 UI Consistency Checklist	11
10. API Specification	12
11. Testing Checklist	13
12. Deployment and Running the App	16
13. Appendix	16

1. Team Information

Team size: 3 students. Roles are fixed for collaboration.

Member	Student ID	Role	Main Responsibilities
Peerawit Yawirach	682115032	UI	Client pages (views/client), client static assets (public/client)
Tingfeng Wang	682115509	API	Models + services + storage (src/models, src/services, src/data)+database(s upabase postgre sql)
Netipong Saiso	682115023	Admin	Controllers + admin pages + integration; integrator for shared files

2. Product Vision

Write 3–5 sentences describing the product goal and value (who, problem, why).

(E.g., This product helps customers pre-order bakery items online instead of ordering through chat. Customers often miss product options, prices, or availability when ordering manually. The web app makes ordering clear and consistent with product pages, a cart, and checkout. Admins can add new bakery items and update order statuses (e.g., pending, confirmed, ready).)

Vision statement: make a general showcase of our brand and products. Let the customer know our concept, inspiration, design, and brand image. And also the customer can order the goods directly in our shop. To improve our brand visibility, enhance customer engagement, and create a convenient online shopping experience that connects customers with our fashion style and values. Through our website, we aim to build trust with customers, present our collections clearly, and make it easy for users to explore, discover, and purchase our products anytime and anywhere.

Target users: People who know our brand and loves fashion things and people who wanted to see some latest design.

3. Scope and MVP Requirements

Your team must implement the MVP flows below. Keep the scope simple but complete end-to-end.

3.1 Customer Features (MVP)

C0. Identify customer (simple)

- ~~Option A (recommended): Choose user from seed users (no password)~~
- Option B: Basic login (username/password)

C1. Browse products

- Product list page
- Product detail page

C2. Search

- Keyword search/filter

C3. Cart

- Add to cart
- View/update cart (change qty / remove)

C4. Checkout → Create Order

- Checkout creates an order in the backend
- Payment is simulated ("Pay now" → marks order as PAID)
- Show the order confirmation page.

3.2 Admin Features (MVP)

A1. Admin login

- Simple admin login is OK (hardcoded admin or simple auth)

A2. Product management

- Create / edit / delete products

A3. Order management

- View orders (order id, customer, total, created date)

3.3 Optional Feature (Choose exactly 1)

Pick ONE bonus feature (keep it small): wishlist, ratings, discount code, sort-by-price, featured items, etc.

Chosen optional feature: featured items

3.4 Out of Scope (Not required)

Real payment gateway, password reset/email verification, complex roles/permissions, inventory tracking, image upload (use image URLs).

4. Use Cases

Provide a simple use case diagram (paste image) AND write 3 key use case descriptions.

Use case diagram: (paste here)

Use Case: Browse products

Primary actor	visitor/customer
Preconditions	Web service is available;with real data in the database
Trigger	User open the page and click the product
Main flow	The customer opens the shop website. The system displays product categories such as men, women, and kids. The customer chooses a category or directly browses all products. The system shows a list of clothing products. The customer views product information, including image, name, price, and category. The customer clicks one product. The system displays detailed information for the selected product.
Alternative/exception flows	If no products are available, the system shows a message such as “No products found..” If the selected category has no products, the system shows”No products found.”. If the user input the page does not exist, the system displays a 404page.
Postconditions	The customer successfully browses products or product details. The system state remains unchanged.

Use Case: Checkout & create order

Primary actor	Customer already registered
Preconditions	The customer has logged in to the website.

	The shopping cart contains at least one product. The selected products are available in stock. The customer has entered the required checkout information, such as shipping address and contact details.
Trigger	The customer clicks the “add to cart” or “confirm order” button.
Main flow	The customer reviews the items in the shopping cart. The customer proceeds to the checkout page. The system displays the order summary, including selected products, quantities, prices, and total amount. The customer confirms the order. The system validates the order information. The system checks product availability and stock quantity. The system creates a new order record. The system updates product stock. The system clears the purchased items from the shopping cart. The system displays an order success message and order details. The customer select the address then pay The admin system will count this revenue and see the status updated
Alternative/exception flows	If the cart is empty, the system does not allow checkout. If stock is insufficient, the system shows an error and prevents order creation. If customer information is incomplete, the system asks the customer to fill in the missing information. If the database operation fails, the system shows an error message.
Postconditions	The order is stored in the database. Stock is updated. The cart is cleared after successful checkout.

Use Case: Admin manages products

Primary actor	admin
Preconditions	Admin has logged into the system The admin account has valid admin privileges. The system and database are available

	For adding/editing a product, at least one valid product category exists.
Trigger	The admin selects Product Management from the admin panel to view, add, edit, or delete products.
Main flow	Admin opens the Product Management page. System displays the product list, including search/filter and pagination options Admin chooses one management action: add, edit, or delete a product If adding or editing, the system shows the product form. Admin enters or updates product details such as name, description, price, category, audience, stock quantity, active status, and images. System validates the input data. System saves the new or updated product information and related product images. System redirects the admin back to the product list and displays the updated result.
Alternative/exception flows	If the admin is unauthorized, the system redirects to login. If the input is invalid, the system shows an error and does not save the product. If image upload fails, the system reports the failure. If the target product does not exist, the system redirects back to the product list. If delete is chosen, the system soft-deletes the product.
Postconditions	Product information is updated in the database; the product is created, edited, or soft-deleted; related images are updated; and the product list reflects the latest state.

5. System Architecture (MVC)

Paste a diagram showing MVC separation and major components (client views, controllers, services, storage).

Architecture diagram: (paste here)

5.1 Technology Stack (fixed)

Layer	Technology
Frontend (Views)	EJS + Tailwind CSS

Backend (Server)	Node.js + TypeScript + Express + JWT auth + cloudinary storage
Architecture	MVC
Data Storage	supabase
Deployment	Render (ejs cannot render in frontend platform like vercel)

5.2 Tools

Design: Figma / diagrams (specify) **Figma**

Implementation: VS Code, Node.js, npm, GitHub

Project management: Trello / Notion / GitHub Projects (choose one) **Github**

6. Software Process Model

Choose ONE: ☐ Waterfall ☐ Iterative ☒ Agile (lightweight) ☐ Prototyping

Reason(s) for your choice (3–5 bullets):

As a demo fashion shopping website, a MVP product, agile is the best process model to choose.

Agile models can handle fastest structure building, define the basic requirement and frequent changes after improving the whole functions. And even the short sprints, frequent feedback, and incremental development. When the basic design and client pages ended, we found that so much elements we still have to add like addresses, banner settings, update the stock management logic something like that. With agile we can do that real quick and test it frequently.

7. Project Timeline (3 Weeks)

Fill the table with your plan. Keep tasks small and assign an owner.

Task	Start date	End date	Owner (UI/API/Admin)	Status (Planned/Doing/Done)
Making Figma	27/02/2026	27/02/2026	API	Done
Develop API and design the data structure	28/02/2026	03/03/2026	API	Done
Admin UI	01/03/2026	03/03/2026	UI	Done
Client UI	01/03/2026	04/03/2026	UI	Done
Testing	02/03/2026	06/03/2026	Admin	Done

Improve the UX	06/03/2026	08/03/2026	Admin	Done
Deploy	09/03/2026	09/03/2026	API	Done

8. Git Collaboration Rules (No PR)

Your team uses ONE shared repository and works on main. Pull before coding, commit small, push often.

Rule	Description
Pull before coding	Run git pull every time before you start working.
Folder ownership	UI: views/client & public/client; API: src/services/models/data; Admin: src/controllers + admin views; avoid editing other zones.
Shared files	src/app.ts, src/routes/*, views/partials/*, public/css/*, package.json — claim in chat before editing (recommended: Admin integrates).
Commit & push often	Commit every 30–60 minutes or per feature; use clear messages like 'UI: cart page'.

9. UI Design (Lightweight)

Purpose: Align the team on screens, layout, and interaction behavior before implementation. Keep this section short (1–2 pages).

UI Design (Lightweight)

The purpose of this UI design section is to align the team on the main screens, overall layout, and key interaction behavior before implementation. The design focuses on a simple, modern, and user-friendly shopping experience for customers, while also providing a clear and efficient management interface for administrators.

Design Goal

The system is designed as an online clothing store with two main interfaces: a client-facing shopping interface and an admin management interface. The client side should feel clean, visual, and easy to navigate, while the admin side should support quick product and order management with minimal complexity. The team agreed to keep the design consistent across screens by using a common layout structure, clear navigation, and predictable user interactions.

Key Screens

- Home Page: Displays promotional banners, featured categories, and highlighted products to help users start browsing quickly.
- Product Listing Page: Shows products in a grid layout with search, category filters, audience filters, sorting, and pagination.
- Product Detail Page: Presents detailed product information, multiple product images, price, stock availability, and actions such as add to cart or wishlist.
- Cart and Checkout Pages: Allow users to review selected products, update quantities, choose an address, and place an order.
- User Account Pages: Let users manage their profile, saved addresses, order history, and wishlist.
- Admin Dashboard: Provides a quick overview of the system, including access to products, orders, and customer management.
- Admin Product Management Page: Allows admins to search, filter, add, edit, delete, activate/deactivate, and upload images for products.

Layout and Navigation

The client interface follows a standard e-commerce layout with a header, main content area, and footer. The header includes the logo, navigation menu, search, cart, and user account access. Product-related pages use card-based layouts and clear call-to-action buttons so users can browse and shop easily.

The admin interface uses a sidebar and topbar layout. The sidebar gives direct access to dashboard, products, orders, customers, and settings. Product forms use a simple single-column structure to reduce input errors and improve readability.

Interaction Behavior

- Users can browse products by category, audience, and keyword search.
- Selecting a product opens the product detail page.
- Product filters and search help users narrow down results quickly.
- Form validation is shown when required fields are missing or invalid.
- Successful actions such as saving a product or placing an order return the user to the relevant page with updated information.
- In the admin product page, image upload provides preview feedback before saving.
- Deleting a product requires confirmation to avoid accidental removal.
- Active and inactive product states help admins control product visibility in the store.

Visual Style and Consistency

The visual style is clean and modern, suitable for a fashion retail website. Large product images, simple typography, and balanced spacing are used to keep the interface readable and attractive. The team aims to maintain consistent button styles, table layouts, form fields, and spacing across both client and admin pages. Important actions are highlighted with stronger contrast, while warning or destructive actions are visually distinguished.

Responsive Considerations

The design supports both desktop and mobile usage. On smaller screens, navigation becomes more compact, product grids display fewer columns, and content stacks vertically for easier

reading. Admin tables may use horizontal scrolling on mobile devices. Buttons and form inputs are designed to remain touch-friendly across screen sizes.

9.1 Low-Fidelity Wireframes

Provide simple wireframes (boxes/labels). Include at least:

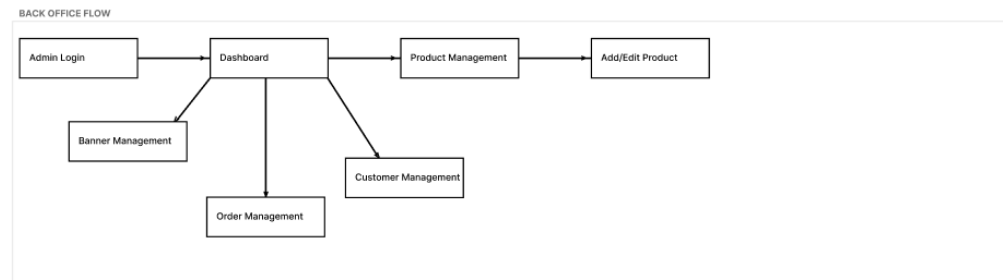
- 1 wireframe for the Customer flow

(Customer: Browse → Product Detail → Cart → Checkout → Order History)



- 1 wireframe for the Admin flow

(Admin: Admin Login → Manage Products → Manage Orders (Update Status))



You can use paper sketches, PowerPoint, Figma, or diagrams. Insert screenshots here.

9.2 Logo and Color theme

- Logo design

Specify logo design here (E.g., text only, icon (free icon) + text or custom logo)

Text only design

- Color theme

Specify color theme here including primary color, secondary color, color for text, status colors)

Monochrome color scheme for all elements

9.3 UI Consistency Checklist

Tick each item after implementing the UI:

- ☒ Consistent header/navigation across pages

- ☑ Consistent button styles (primary/secondary)
- ☑ Consistent form layout + validation messages
- ☑ Loading / empty / error states are shown (at least 1 example)

- ☑ Responsive layout works on mobile width (e.g., 390px) and desktop (Optional)

10. API Specification

List your endpoints. Keep them consistent with your implementation.

Client / Public Endpoints

Endpoint	Method	Request (params/body)	Response (JSON)	Notes
/	GET	none	HTML client/home	Public homepage with banners and featured products
/login	GET	none	HTML client/login or 302 -> /	Redirects home if already logged in
/login	POST	b: email, password, [redirect]	302 -> redirect path or /; on failure HTML client/login	Sets auth cookie
/register	GET	none	HTML client/register or 302 -> /	Redirects home if already logged in
/register	POST	b: full_name, email, password, [phone]	302 -> /; on failure HTML client/register	Creates client account and logs user in

/logout	GET	none	302 -> /	Clears auth cookie and destroys session
/products	GET	q: page, category, category_name, q, audience, sort	HTML client/products	Product listing with filters and pagination
/products/:id	GET	p: id; q: success, error	HTML client/productDetail or 404 HTML	Increments product click count with throttle
/cart	GET	q: error	HTML client/cart	Public page; guest sees empty/null cart
/cart/add	POST	b: product_id, quantity, [size]	302 -> /cart	Auth
/cart/update	POST	b: id or id[], quantity or quantity[]	302 -> /cart	Auth; qty <= 0 removes item
/cart/remove/:id	POST	p: id	302 -> /cart	Auth
/wishlist	GET	q: success, error	HTML client/wishlist	Auth
/wishlist/add	POST	b: product_id	302 -> referer or /wishlist	Auth; appends success/error query
/wishlist/remove/:id	POST	p: id	302 -> /wishlist?success=...	Auth
/orders	POST	none	302 -> /orders/:id or 302 -> /cart?error=...	Auth; creates order from current cart
/orders/:id/confirm-pay	POST	p: id; b: address_id	302 -> /order-confirm/:id?payment=success or back to order detail with error	Auth
/order-confirm/:id	GET	p: id; q: payment	HTML client/orderConfirmation or redirect	Auth; only for paid orders

/orders	GET	q: page	HTML client/orders	Auth; paginated order history
/orders/:id	GET	p: id; q: success, error	HTML client/orderDetail or 404 HTML	Auth
/profile	GET	q: success, error	HTML client/profile	Auth
/profile/edit	GET	q: success, error	HTML client/editProfile	Auth
/profile/edit	POST	b: full_name, phone	302 -> /profile; on failure HTML client/editProfile	Auth; refreshes auth cookie
/profile/addresses	GET	q: success, error	HTML client/addressBo ok	Auth
/profile/change-p assword	GET	none	HTML client/changePass word	Auth
/profile/change-p assword	POST	b: current_passwor d, new_password, confirm_passwor d	HTML client/changePass word	Auth; returns success/error on same page
/addresses	POST	b: label, full_name, phone, address_line1, address_line2, city, state, postal_code, country, is_default	302 -> /profile/addresses?success=... or ?error=...	Auth
/addresses/:id/edi t	POST	p: id; same body as add address	302 -> /profile/addresses?success=... or ?error=...	Auth
/addresses/:id/del ete	POST	p: id	302 -> /profile/addresses?success=...	Auth

Admin Endpoints

Endpoint	Method	Request (params/body)	Response (JSON)	Notes
/admin/login	GET	none	HTML admin/login or 302 -> /admin	Public admin login page
/admin/login	POST	b: email, password	302 -> /admin; on failure 401 HTML admin/login	Admin authentication
/admin/logout	GET	none	302 -> /admin/login	Admin; clears auth cookie and session
/admin	GET	none	HTML admin/dashboard	Admin dashboard
/admin/products	GET	q: page, q, category_id, active, audience	HTML admin/products	Admin product list with filters and pagination
/admin/products/add	GET	none	HTML admin/productAdd	Admin product create form
/admin/products/add	POST	b: name, description, composition_care, price, category_id, stock_quantity, is_active, image_urls	302 -> /admin/products; on failure HTML admin/productAdd	Admin; creates product and saves images
/admin/products/:id/edit	GET	p: id	HTML admin/productAdd or 302 -> /admin/products	Admin
/admin/products/:id/edit	POST	p: id; same body as add product	302 -> /admin/products or 302 -> /admin/products?error=...	Admin; overwrites image list

/admin/products/:id/delete	POST	p: id	302 -> /admin/products	Admin; soft delete via del_flg=true
/admin/uploads/product-image	POST	b(JSON): file (base64/data URL)	200 JSON { "message": "upload success", "url": "..."} or 400 JSON { "message": "..."} }	Admin; uploads image to Cloudinary
/admin/orders	GET	q: order_no, customer, status, date_from, date_to, min_total, max_total	HTML admin/orders	Admin order search/filter
/admin/orders/:id	GET	p: id	HTML admin/orderDetail or 302 -> /admin/orders	Admin
/admin/orders/:id/status	POST	p: id; b: status	302 -> /admin/orders/:id or back with error query	Admin; updates order status
/admin/customers	GET	q: q	HTML admin/customers	Admin customer search
/admin/customers/:id	GET	p: id	HTML admin/customerDetail or 302 -> /admin/customers	Admin
/admin/settings/banner	GET	q: kind	HTML admin/bannerSettings	Admin; kind = home or category
/admin/settings/banner/add	GET	q: kind	HTML admin/bannerAdd	Admin
/admin/settings/banner/add	POST	b: title, description, image_url, product_id, banner_kind, audience_scope, active_from,	302 -> /admin/settings/banner?...; on failure HTML admin/bannerAdd	Admin

		active_to, is_active		
/admin/settings/ banner/:id/edit	GET	p: id	HTML admin/bannerAd d or redirect	Admin
/admin/settings/ banner/:id/edit	POST	p: id; same body as add banner	302 -> /admin/settings/ banner?...; on failure HTML admin/bannerAd d	Admin
/admin/settings/ banner/:id/delete	POST	p: id	302 -> /admin/settings/ banner?...	Admin; soft delete
/admin/settings/c ategories	GET	none	HTML admin/categoryS ettings	Admin category list/create page
/admin/settings/c ategories	POST	b: name, slug, audience	302 -> /admin/settings/c ategories; on failure HTML admin/categoryS ettings	Admin
/admin/settings/c ategories/:id/edit	GET	p: id	HTML admin/categoryS ettings or redirect	Admin
/admin/settings/c ategories/:id/edit	POST	p: id; b: name, slug, audience	302 -> /admin/settings/c ategories; on failure HTML admin/categoryS ettings	Admin; also updates linked product audience
/admin/settings/c ategories/:id/dele te	POST	p: id	302 -> /admin/settings/c ategories	Admin; clears linked category_id, then soft deletes category

11. Testing Checklist

List at least 8 UAT tests: 5 happy paths + 3 edge cases.

Test ID	Scenario	Expected result	Pass/Fail
T1	Create the account.	Can to login ,Account was created and keep in the superbase	pass
T2	Login to the website.	Can use the data from account and can use all of function	pass
T3	Verify the main category links redirect to the correct department pages.	Test: Click on WOMEN and verify it loads the Women's clothing catalog. Test: Click on MEN and verify it loads the Men's clothing catalog. Test: Click on KIDS and verify it loads the Kids' clothing catalog.	pass
T4	Verify the brand logo redirects the user to the homepage.	Click the MAISON logo from every pages and ensure that it redirect the user back to the home page	pass
T5	Verify The 'Continue Shopping' button redirect user to the product page.	Click the 'Continue Shopping' button from the cart page and verify if it redirect user into product pages or not	pass
T6	Verify the Search Button Takes the user to the products page.	Click Search to ensure that it redirect user into a full products page	pass
T7	Verify the Account button.	Click Account to login page if already login ensure the profile page	pass
T8	Verify the Cart buttons.	Click Cart to redirect user to the cart page	pass
T9	Verify The Search Functionality.	Ensure That the Search Function uses the word to match the products name	pass

T10	Verify the category button	When chooses a category only the products in that particular category will show up	pass
T11	Verify the sort function.	When choose the one sort it will show the sorted product.	pass
T12	Verify the product	When click on the product show the products detail page that have pictures, name, size choice button, Add cart, Add wish, prize, in stock, you may also like and footer	pass
T13	Verify the audience button in the main products page	When select audience type it redirect the user into that audience products page	pass
T14	Verify the wishlist button	When click the wishlist button at the bottom of the screen it redirect the user into the wishlist page	pass
T15	Verify add to wish button.	Show the the wish list add message	pass
T16	Verify shop now button in home page.	When click it will ensure the products detail page	
T17	Verify 'Shop the edit' button.	When click shop the edit button, it redirect the user into that particular product detail page	pass
T18	Verify EXPLORE COLLECTION button.	When click the button ensure Products category	pass
T19	Verify the pages button	When click the numbered pages button it redirect the the user into that particular page on that particular	pass

		products category and audience	
T20	Verify go to page function.	When click the number it can type or choose the number of product page and when the number be 0 or more than the product page return the cannot go message when click go, If it correct way it will go to the page that you choose.	pass
T21	Verify the orders button	When click the orders button at the bottom of the screen it redirect user into the orders page	pass
T22	Verify if the 'Start Shopping' button redirect user into products page	When click the 'Start Shopping' button it will redirect user into the products page	pass

12. Deployment and Running the App

GitHub Repository URL:

<https://github.com/camt-pathathai/953261-953202-term-project-alphamonno1.git>

Deployed URL: <https://clothprojectrender.onrender.com> (/admin is the administrator system)

13. Appendix

Other diagrams, screen shots of project plan tool, and any additional notes.